

DOM manipulation Techniques used in HTML

1. Selecting Elements

Selecting elements is the first step in DOM manipulation. You can select elements using various methods:

- `document.getElementById(id)`: Selects an element by its ID.

```
let element = document.getElementById("myId");
```

- `document.getElementsByClassName(className)`: Selects elements by their class name.

```
let elements = document.getElementsByClassName("myClass");
```

- `document.getElementsByTagName(tagName)`: Selects elements by their tag name.

```
let elements = document.getElementsByTagName("p");
```

- `document.querySelector(selector)`: Selects the first element that matches a CSS selector.

```
let element = document.querySelector(".myClass");
```

- `document.querySelectorAll(selector)`: Selects all elements that match a CSS selector.

```
let elements = document.querySelectorAll(".myClass");
```

2. Changing Content

You can change the content of an element using properties like `innerHTML`, `textContent`, and `innerText`.

- `element.innerHTML`: Sets or gets the HTML content of an element.

```
element.innerHTML = "<strong>New Content</strong>";
```

- `element.textContent`: Sets or gets the text content of an element.

```
element.textContent = "New Content";
```

- `element.innerText`: Similar to `textContent`, but it respects the styling (e.g., `display: none`).

```
element.innerText = "New Content";
```

3. Changing Attributes

You can change the attributes of an element using methods like `setAttribute`, `getAttribute`, and `removeAttribute`.

- `element.setAttribute(name, value)`: Sets the value of an attribute.

```
element.setAttribute("src", "image.jpg");
```

- `element.getAttribute(name)`: Gets the value of an attribute.

```
let src = element.getAttribute("src");
```

- `element.removeAttribute(name)`: Removes an attribute.

```
element.removeAttribute("src");
```

4. Changing Styles

You can change the styles of an element using the `style` property.

- `element.style.property`: Sets the value of a CSS property.

```
element.style.color = "red";  
element.style.backgroundColor = "yellow";
```

5. Adding and Removing Classes

You can add or remove classes using the `classList` property.

- `element.classList.add(className)`: Adds a class to the element.

```
element.classList.add("newClass");
```

- `element.classList.remove(className)`: Removes a class from the element.

```
element.classList.remove("oldClass");
```

- `element.classList.toggle(className)`: Toggles a class on the element.

```
element.classList.toggle("active");
```

6. Creating and Removing Elements

You can create new elements and remove existing ones.

- `document.createElement(tagName)`: Creates a new element.

```
let newElement = document.createElement("div");
```

- `parentElement.appendChild(newElement)`: Appends a new element as a child.

```
parentElement.appendChild(newElement);
```

- `parentElement.removeChild(childElement)`: Removes a child element.

```
parentElement.removeChild(childElement);
```

7. Event Handling

You can add event listeners to elements to handle user interactions.

- `element.addEventListener(event, function)`: Adds an event listener to an element.

```
element.addEventListener("click", function() {  
    alert("Element clicked!");  
});
```

8. Traversing the DOM

You can navigate through the DOM tree to find related elements.

- `element.parentNode`: Gets the parent node of an element.

```
let parent = element.parentNode;
```

- `element.childNodes`: Gets the child nodes of an element.

```
let children = element.childNodes;
```

- `element.firstChild`: Gets the first child node of an element.

```
let firstChild = element.firstChild;
```

- `element.lastChild`: Gets the last child node of an element.

```
let lastChild = element.lastChild;
```

- `element.nextSibling`: Gets the next sibling of an element.

```
let nextSibling = element.nextSibling;
```

- `element.previousSibling`: Gets the previous sibling of an element.

```
let previousSibling = element.previousSibling;
```

By mastering these DOM manipulation techniques, you can create dynamic and interactive web pages that respond to user actions and provide a better user experience.